

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 9

Student Name: Gorapalli Sravya

Roll No.: A24126552257

Date: 26-08-2026

1. TITLE

Develop a Student Profile Manager Using JavaScript Class and DOM

2. AIM

To develop a Student Profile Manager web application using a JavaScript class to model student data, and DOM manipulation to add, display and delete student profiles dynamically.

3. OBJECTIVE

The objectives of this experiment are:

- To define a JavaScript class (Student) with properties and a method.
- To create multiple objects (instances) of the class from form input.
- To dynamically render a list of objects onto the page using the DOM.
- To remove an object from the underlying array and re-render the list.

4. THEORY

JavaScript ES6 introduced the class syntax as a cleaner way to implement object-oriented concepts. A class is defined using the class keyword, with a constructor() method that initializes an object's properties, and additional methods that define its behaviour.

Objects (instances) of a class are created using the new keyword, for example new Student(...). Each instance has its own copy of the properties defined in the constructor and can call the methods defined on the class.

Combined with DOM methods such as createElement() and appendChild(), classes can be used to model structured data (such as a student) and dynamically render a list of such objects on a webpage, updating the list whenever the underlying array changes.

5. ALGORITHM / PROCEDURE

1. Create an HTML file with input fields for name, roll number and branch, an Add button, and a container for the profile list.
2. Define a Student class with a constructor that accepts name, rollNo and branch.
3. Add a getSummary() method to the class that returns a formatted string.

4. Create an array, students, to hold Student objects.
5. Write an addStudent() function that reads the input fields, creates a new Student object using new, and pushes it into the array.
6. Write a renderStudents() function that clears and rebuilds the profile list from the students array using the DOM.
7. Add a Delete button to each rendered profile that removes that student from the array (using splice) and re-renders the list.
8. Attach addStudent() to the Add button using addEventListener().
9. Save the file and test adding and deleting multiple student profiles in the browser.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Profile Manager</title>
</head>
<body>
  <h1>Student Profile Manager</h1>

  <input type="text" id="nameInput" placeholder="Name">
  <input type="text" id="rollInput" placeholder="Roll No">
  <input type="text" id="branchInput" placeholder="Branch">
  <button id="addBtn">Add Student</button>

  <div id="profileList"></div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
class Student {
  constructor(name, rollNo, branch) {
    this.name = name;
    this.rollNo = rollNo;
    this.branch = branch;
  }

  getSummary() {
    return `${this.name} (${this.rollNo}) - ${this.branch}`;
  }
}

const students = [];

function renderStudents() {
  const listDiv = document.getElementById("profileList");
  listDiv.innerHTML = "";

  students.forEach((student, index) => {
    const card = document.createElement("div");
```

```

    card.className = "profile-card";

    const text = document.createElement("span");
    text.textContent = student.getSummary();

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.onclick = () => {
        students.splice(index, 1);
        renderStudents();
    };

    card.appendChild(text);
    card.appendChild(deleteBtn);
    listDiv.appendChild(card);
  });
}

function addStudent() {
  const nameField = document.getElementById("nameInput");
  const rollField = document.getElementById("rollInput");
  const branchField = document.getElementById("branchInput");

  const name = nameField.value.trim();
  const rollNo = rollField.value.trim();
  const branch = branchField.value.trim();

  if (name && rollNo && branch) {
    const student = new Student(name, rollNo, branch);
    students.push(student);
    renderStudents();

    nameField.value = "";
    rollField.value = "";
    branchField.value = "";
  }
}

document.getElementById("addBtn").addEventListener("click", addStudent);

```

7. CODE EXPLANATION

Tag / Code	Explanation
class Student { ... }	Defines a blueprint for a student object with properties and a method
constructor(name, rollNo, branch)	Initializes a new Student object's properties
getSummary()	Instance method that returns a formatted summary string
new Student(...)	Creates a new instance (object) of the Student class
students.push() / splice()	Adds a student to, or removes a student from, the array
document.createElement()	Dynamically creates a new profile card / button element
renderStudents()	Rebuilds the visible profile list from the current students array

8. TEST CASES AND EXECUTION DATA

The application was tested in the browser by adding and deleting several student profiles.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Add a student with valid details	A new profile card appears in the list	Profile card appeared correctly	Pass
TC-02	Add a student with an empty field	No profile is added	No profile added	Pass
TC-03	Add three students	All three profiles are listed correctly	All profiles listed correctly	Pass
TC-04	Click Delete on the middle profile	That profile is removed; the others remain	Correct profile removed	Pass
TC-05	Check getSummary() output	Displays name, roll number and branch in the expected format	Correct format displayed	Pass

9. OUTPUT

Output: Student Profiles Added and Removed

Student Profile Manager

Gorapalli Sravya (A24126552257) - CSE (AI & ML)

Rahul Kumar (A24126552210) - CSE

Priya Sharma (A24126552233) - ECE

Three student profiles added using the Student class

Student Profile Manager

Name	Roll No
Branch	
Add Student	
Gorapalli Sravya (A24126552257) - CSE (AI & ML) Delete	
Priya Sharma (A24126552233) - ECE Delete	

Profile list after deleting the second student (Rahul Kumar)

10. RESULT

Thus, a Student Profile Manager was successfully developed using a JavaScript class to model student data, and DOM manipulation was used to dynamically add, display and delete student profiles.